KONVEYOR

Adopting Lightspeed's DeepAgents runtime in Konveyor

November 11, 2026

Runtime integration

- Konveyor with Goose
- The prototype with Lightspeed
- Changes in Lightspeed's runtime
- Coolstore planning demo
- Questions

Konveyor with Goose

1. **You choose an app and a task**

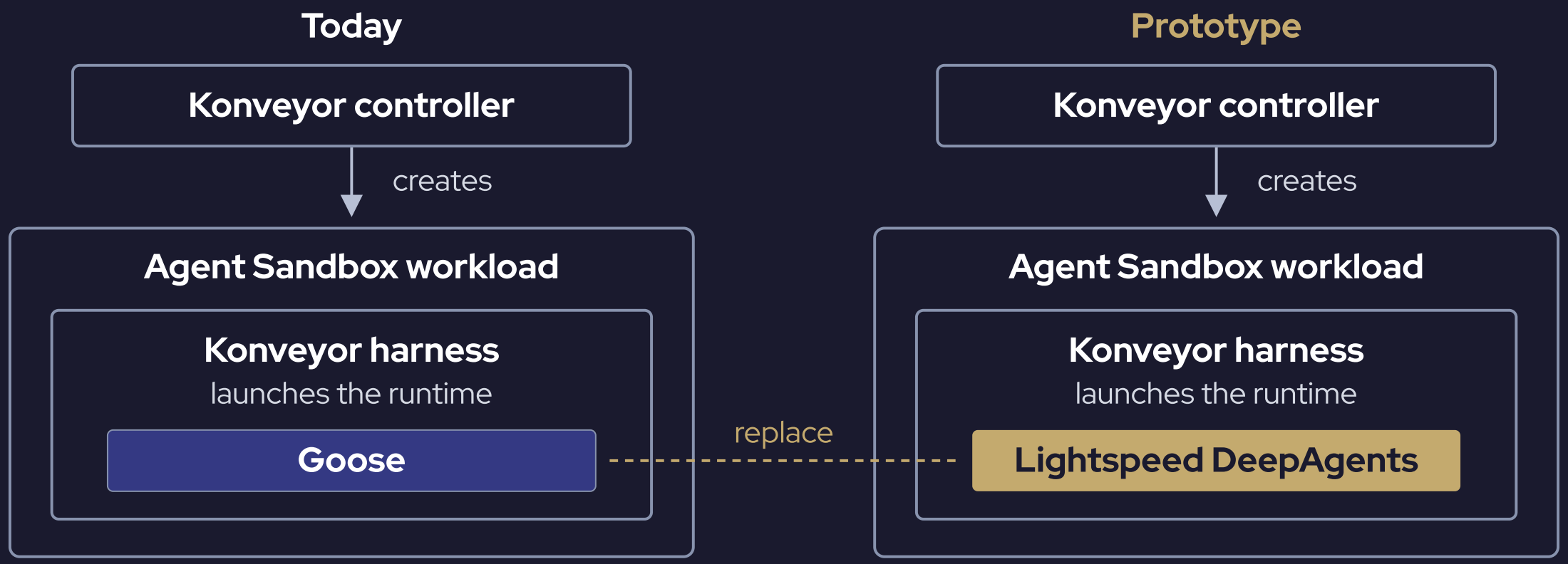
2. **Our harness prepares the repo**

3. **Goose works on the code**
   Calls the model, uses tools, commits locally

4. **Our harness verifies and pushes**
   You get the result on a Git branch



Konveyor with Lightspeed



Our harness still prepares the repo, verifies the work and pushes to Git.

Keeping the conversation for a second prompt

What it does: lets a later prompt use the conversation and tool results from an earlier prompt.

What we changed: Lightspeed created and closed its execution state for every query. We added a session that keeps that state until it is closed.

Why Konveyor needs it: when a run reaches its turn limit, our harness must ask the agent to write a handoff before verifying and pushing its work.

Components: DeepAgents provider, RuntimeSession

Asking the user before acting

What it does: lets the agent request permission to use a tool or ask the user a question.

What we changed: added separate runtime callbacks for permission and questions. The application running the session receives each request and returns the user's answer or decision.

Why Konveyor needs it: users already approve commands and answer questions in Konveyor. Switching runtimes must preserve both interactions.

Components: tool permission middleware, question callback

Connecting Lightspeed to Konveyor's UI

What it does: carries messages, approvals, questions, guidance and cancellation between Konveyor's UI and the runtime.

What we changed: added an optional ACP adapter around the session. It plugs into the Go harness connection that our UI already uses.

Why Konveyor needs it: users need to watch and guide Lightspeed runs through Konveyor. Keeping the adapter separate lets other hosts use sessions without adopting ACP.

Components: ACP adapter, session core

Guiding or stopping active work

What it does: lets a user send new instructions while the agent works, or stop its current work.

What we changed: guidance reaches the next model call. Cancellation waits for active work to stop and ends local shell commands, while keeping the conversation available.

Why Konveyor needs it: users need to correct the agent or stop an unhelpful run. The harness still needs time to save and push committed work and clean up.

Components: session controls, local shell execution

Reporting usage and reserving a handoff

What it does: reports token usage during a run and signals when the agent reaches its allowed number of model calls.

What we changed: upstream returned usage totals at the end. We added updates after model calls and an explicit turn counter.

Why Konveyor needs it: the UI can show usage while work happens. The harness can reserve some turns to ask for a handoff before the run ends.

Components: usage callback, session turn counter, Konveyor harness

Checking what tools return

What it does: checks text returned by tools for instructions that could redirect the agent, before the model or UI receives it.

What we changed: kept Lightspeed's existing checks in the new session and UI path. Answers from our `ask_user` tool are treated as the user's instructions.

Why Konveyor needs it: application files and MCP results can contain instructions supplied by others. Interactive runs need the same tool-output checks as batch runs.

Components: tool-output inspector, output delivery to the model and UI

Applying the model settings we choose

What it does: applies the chosen reasoning effort and maximum response length when calling the model.

What we changed: the provider adapter was dropping our reasoning-effort choice. We now pass it through and allow an optional limit on the main model's response length.

Why Konveyor needs it: for the GLM demo, we wanted less reasoning and more of the response allowance left for the plan. Those choices have to reach the model.

Components: model/provider adapter, main-model configuration

Detecting a response that was cut short

What it does: recognizes when the provider says it stopped because the response reached its token limit.

What we changed: return an error before running tools from that unfinished response. Keep the token usage already reported.

Why Konveyor needs it: an unfinished answer must not look like completed work. For example, the runtime could otherwise say it finished without producing the requested plan.

Component: response-limit check before tool execution

Coolstore planning demo

A Java EE application we want to move to Quarkus

Play the separate demo video (2:44)

Code and result:

- Konveyor harness integration: PR #345 · source changes
- Lightspeed runtime changes: djzager/lightspeed-agentic-sandbox
- Konveyor UI changes: djzager/tackle2-ui
- Reviewed plan: djzager/coolstore, docs/plan.md

Questions

Lightspeed as the execution backend

Today: Konveyor's `AgentRun` reconciler creates a Sandbox directly. Our Go harness owns preparation, agent execution and finalization.

Candidate integration: delegate execution to a Lightspeed `AgenticRun`; its operator owns the SandboxClaim and workload lifecycle.

Adoption bar: per-run image/startup control, complete access to inputs and credentials, and preservation of harness finalization and outcomes.

Agent Sandbox remains underneath Lightspeed's SandboxClaim path. ADR 0021 separates this qualification from runtime reuse.

Roadblock: selecting the workload image

Current: one shared `sandbox-pod-spec` supplies the base image. Agent and AgenticRun have no per-invocation image selector.

- Java EE to Quarkus needs a JDK and Maven in the workload.
- A future Go Migrator needs the Go toolchain.
- Both images must be usable in concurrent runs.

Required: select the image per invocation and start its Konveyor harness.

Possible contract: a per-run image override or template selection, honoring the image's entrypoint. ADR 0021 makes image/startup control a firm requirement.

Problem: workload placement and access

Current: Lightspeed watches runs across namespaces but creates workloads in its operator namespace. Agent/LLMProvider are cluster-scoped; Secrets are namespaced.

Problem: the workload needs Hub's run-scoped inputs/token, model credentials and a reachable ACP endpoint for Konveyor's UI.

Options to qualify: run near Hub and bind provider credentials there, or run centrally and deliver Hub inputs/token with the required network access.

Components: Hub, provider/Secret binding, PodSpecBuilder and workload networking. ADR 0021 leaves placement open.

Problem: harness input and run completion

Current: Lightspeed mounts batch inputs and a Result template under `/input`; its operator uses the Pod and a completed Kubernetes Result to determine success.

Adapter needed: our harness must consume those inputs, prepare the repo, run the agent, verify/push, clean up and publish the backend Result before exiting.

Why: an agent finishing its answer is earlier than the workload finishing.

If push succeeds but Result publication fails, preserve the work and report the integration failure. ADR 0021 requires that distinction.

Problem: stopping the agent or its workload

Runtime cancellation: stops active prompt/tool work inside the Pod; the Go harness can still finalize.

Operator hard stop: terminates the workload and may prevent handoff, verification, push, token cleanup and Result publication.

Needed: an agreed normal stop path and time to finalize within the workload deadline, with emergency termination handled explicitly.

Discussion: can operator cancellation request harness wind-down before killing the workload? A per-run hard-stop proposal is open in PR #588.

OpenShell roadmap and integration contract

Roadmap question: are you planning OpenShell in this execution path, and on what timeline?

If so: which layer would own workload image/startup, placement, network policy and credential binding?

Integration impact: Hub/Git/model/MCP/ACP access and Kubernetes Result publication need a supported contract under that backend.

Should we qualify today's batch contract, or align on one that remains stable through that change?

ADRs 0016 and 0021 keep OpenShell deferred and preserve today's Gateway selection contract.

Questions